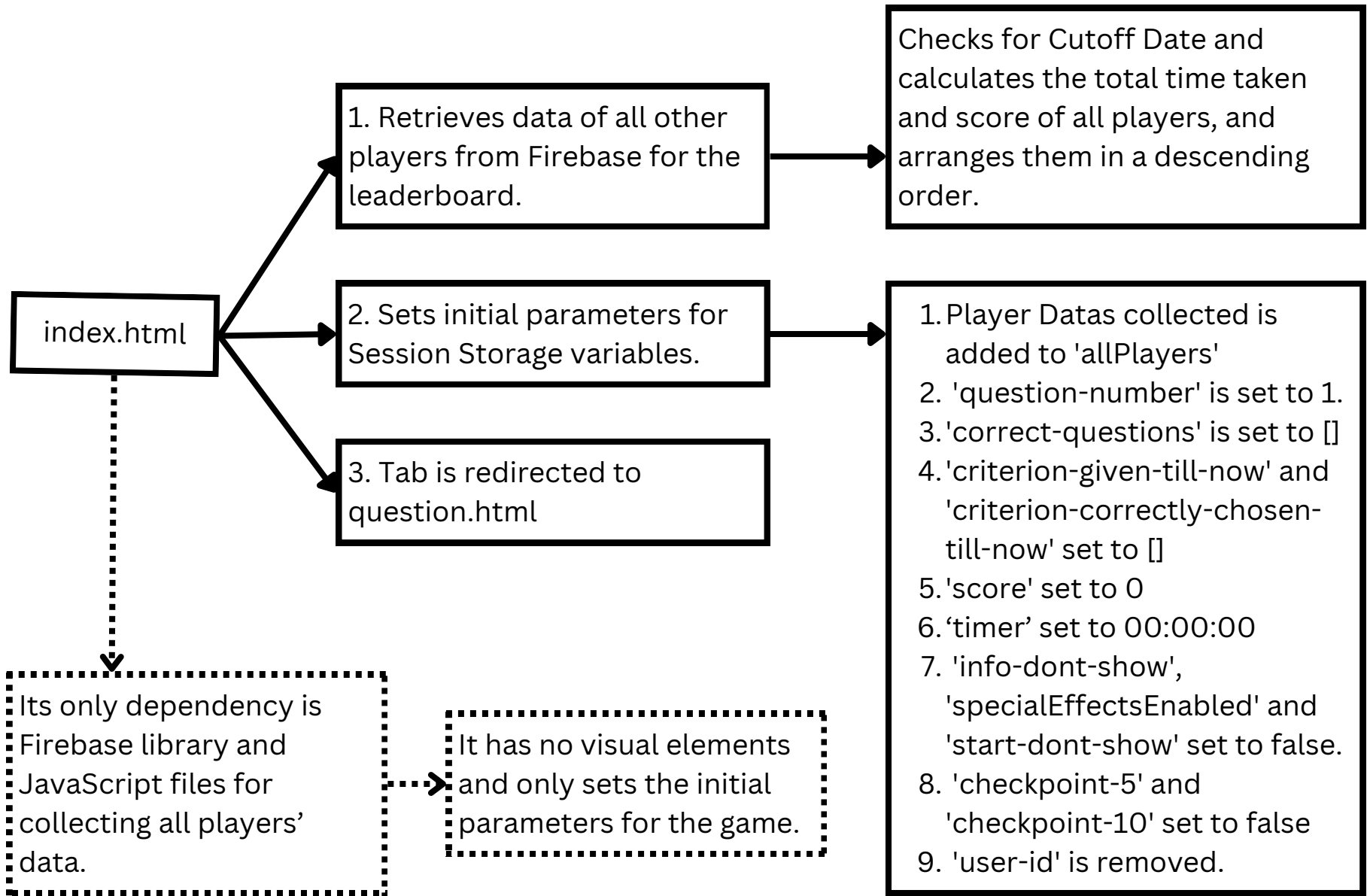
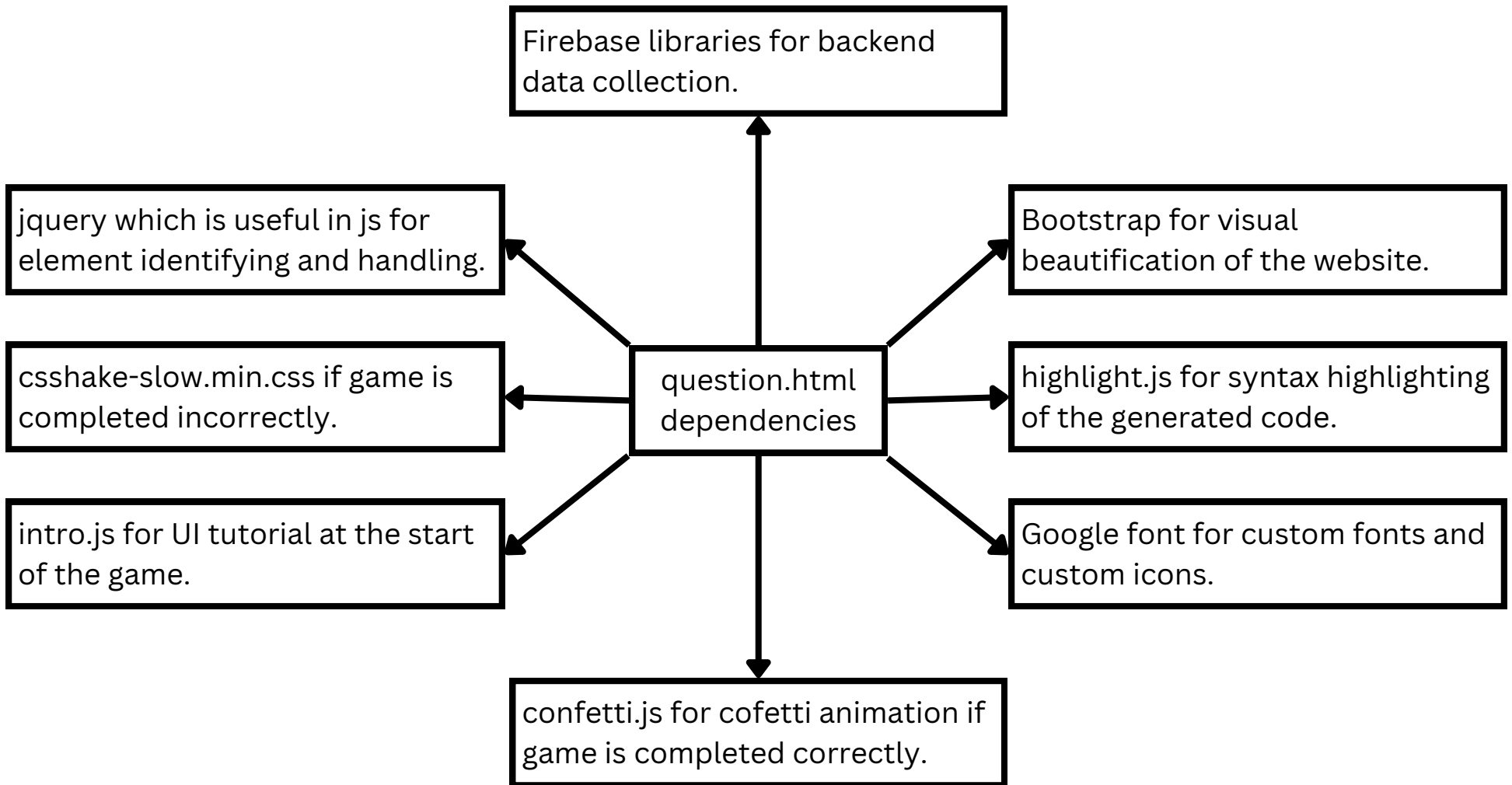
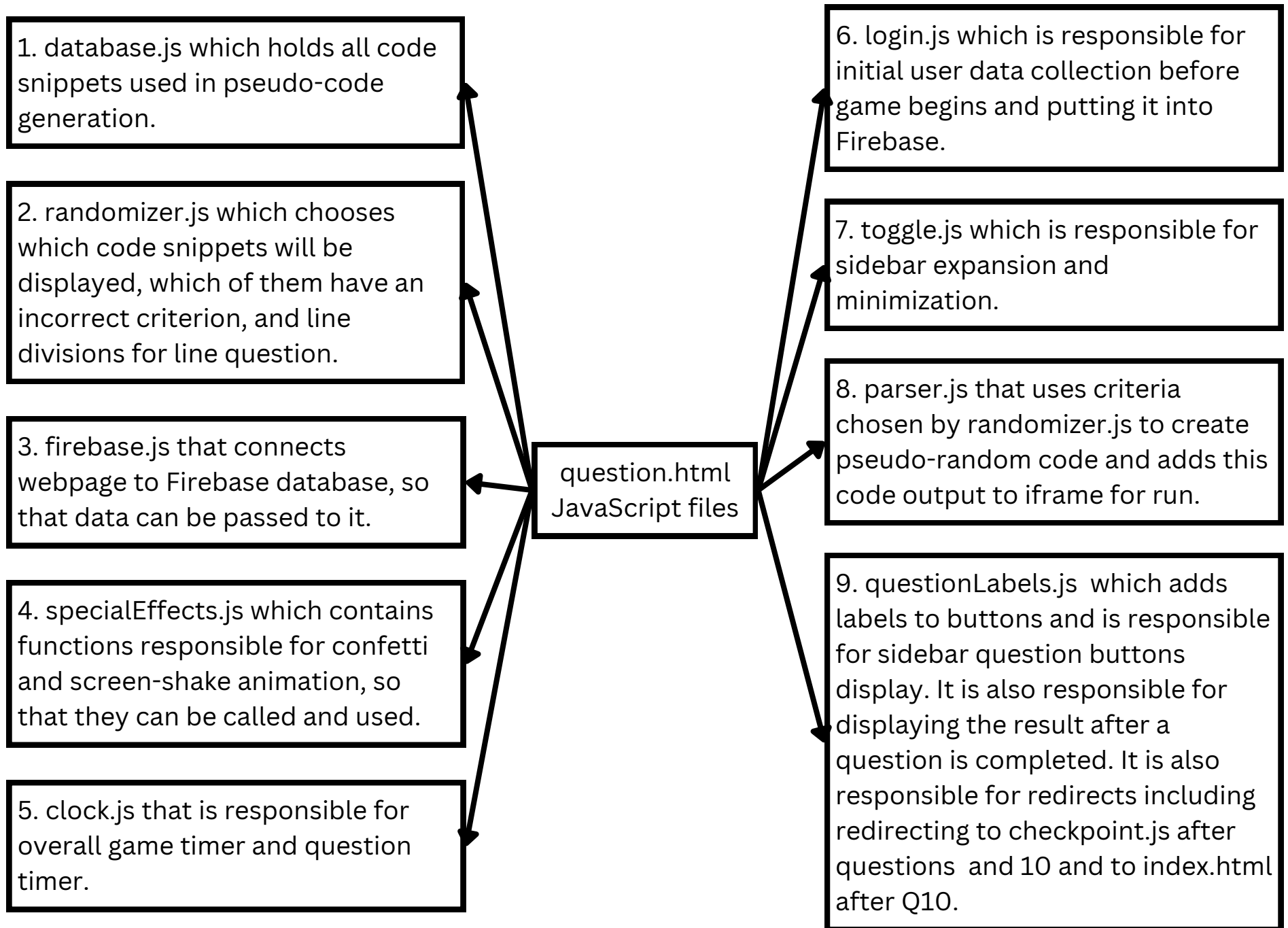








10. miscellaneous.js has important functions related to accessibility and functions responsible for question behaviour on completion.

11. lineQuestion.js defines the webpage's behaviour once user submits the line part of the question.

12. pourQuestion.js defines the webpage's behaviour once user submits the POUR part of the question.

13. criterionQuestion.js defines the webpage's behaviour once user submits the criterion part of the question.

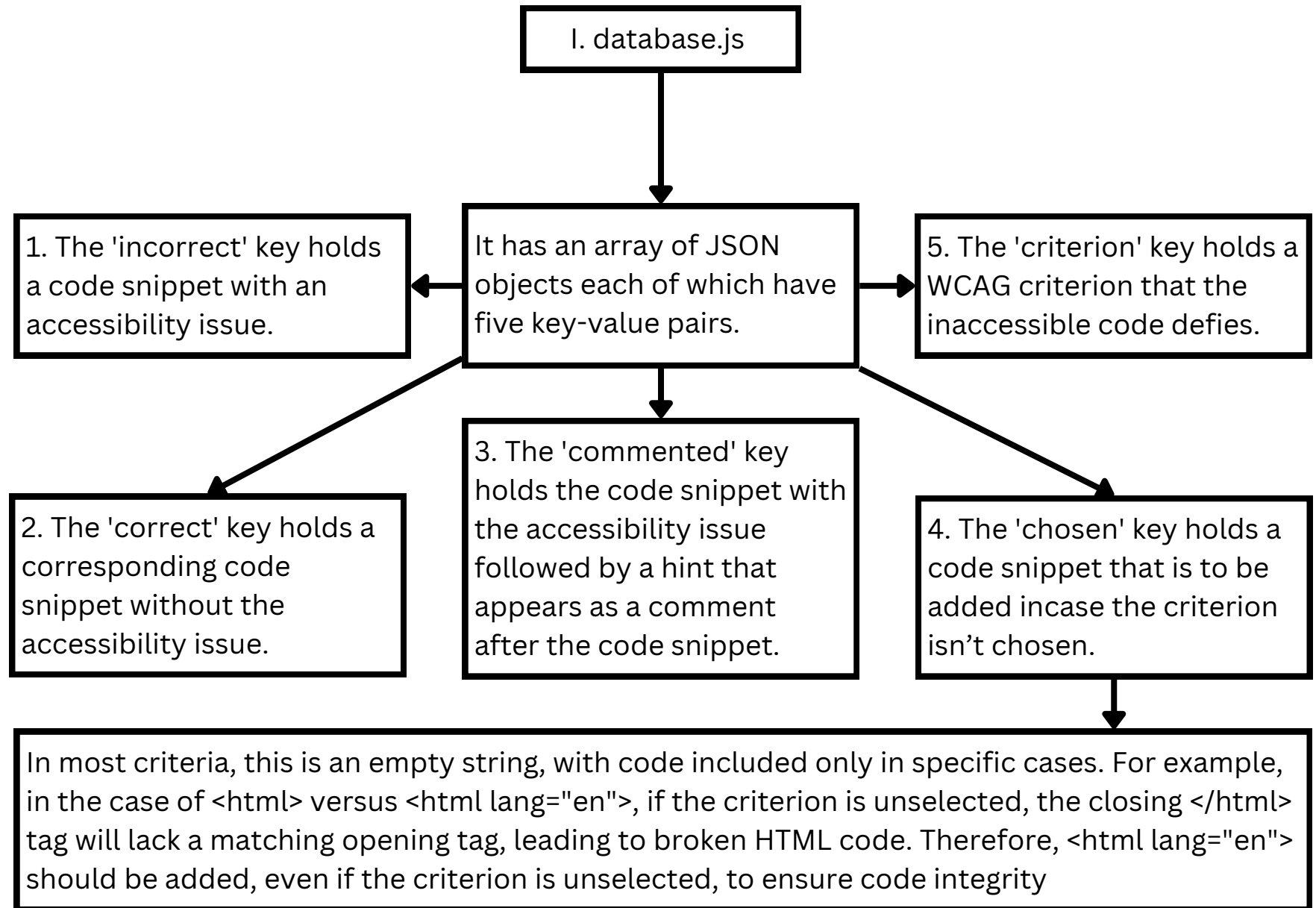
question.html
JavaScript files

14. hint.js defines how the webpage would react if user clicks on the hint button and chooses to take a hint.

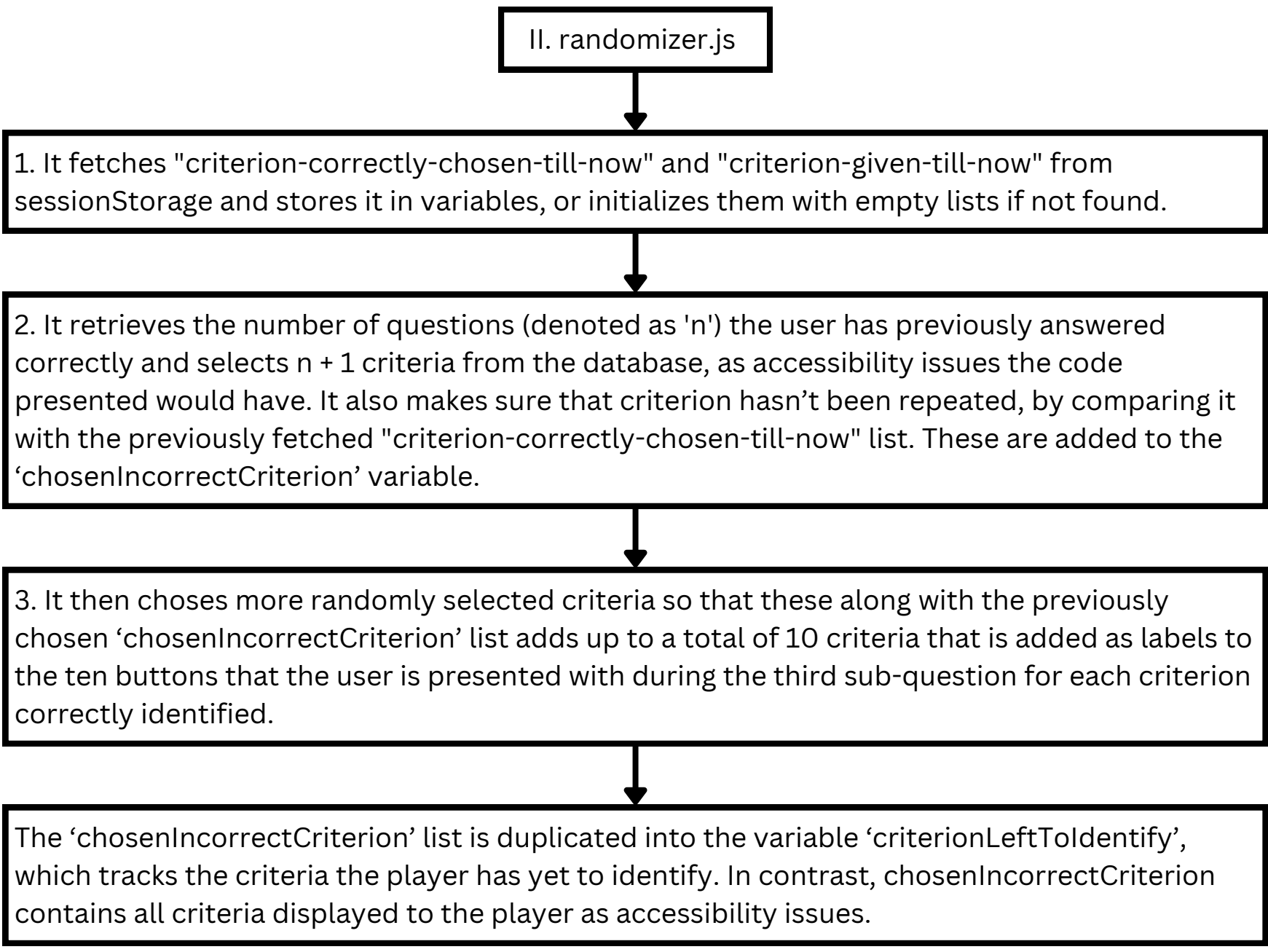
15. leaderboard.js defines how the webpage would react if user opens the leaderboard and if they click on appropriate filters.

16. ranks.js gives user a rank upgrade once they have passed certain milestones, along with congrats message.

17. tutorials.js contains data that will be passed to intro.js library functions for UI tutorial at the start of the game.



II. randomizer.js



```
graph TD; A[II. randomizer.js] --> B[1. It fetches "criterion-correctly-chosen-till-now" and "criterion-given-till-now" from sessionStorage and stores it in variables, or initializes them with empty lists if not found.]; B --> C[2. It retrieves the number of questions (denoted as 'n') the user has previously answered correctly and selects n + 1 criteria from the database, as accessibility issues the code presented would have. It also makes sure that criterion hasn't been repeated, by comparing it with the previously fetched "criterion-correctly-chosen-till-now" list. These are added to the 'chosenIncorrectCriterion' variable.]; C --> D[3. It then chooses more randomly selected criteria so that these along with the previously chosen 'chosenIncorrectCriterion' list adds up to a total of 10 criteria that is added as labels to the ten buttons that the user is presented with during the third sub-question for each criterion correctly identified.]; D --> E[The 'chosenIncorrectCriterion' list is duplicated into the variable 'criterionLeftToIdentify', which tracks the criteria the player has yet to identify. In contrast, chosenIncorrectCriterion contains all criteria displayed to the player as accessibility issues.];
```

1. It fetches "criterion-correctly-chosen-till-now" and "criterion-given-till-now" from sessionStorage and stores it in variables, or initializes them with empty lists if not found.

2. It retrieves the number of questions (denoted as 'n') the user has previously answered correctly and selects $n + 1$ criteria from the database, as accessibility issues the code presented would have. It also makes sure that criterion hasn't been repeated, by comparing it with the previously fetched "criterion-correctly-chosen-till-now" list. These are added to the 'chosenIncorrectCriterion' variable.

3. It then chooses more randomly selected criteria so that these along with the previously chosen 'chosenIncorrectCriterion' list adds up to a total of 10 criteria that is added as labels to the ten buttons that the user is presented with during the third sub-question for each criterion correctly identified.

The 'chosenIncorrectCriterion' list is duplicated into the variable 'criterionLeftToIdentify', which tracks the criteria the player has yet to identify. In contrast, chosenIncorrectCriterion contains all criteria displayed to the player as accessibility issues.

III. firebase.js

1. It has the firebase configuration, initialization and collection name which is used to connect the webpage to firebase DB so that data can be fetched or pushed to firebase cloud DB.

IV. specialEffects.js

1. It fetches "specialEffectsEnabled" from sessionStorage to ensure that if player had turned off special effects in previous question, they would stay off in current question.

2. It contains the confettiAnimation() function which, when called, triggers a celebratory confetti animation on the webpage when the question correctly.

3. It contains the shakeScreen() and shakeElement() functions which, when called, triggers a shake animation to the entire screen (when question is answered incorrectly), or shake animation to question carousel (when a sub-question is answered incorrectly) respectively.

V. clock.js

1. It fetches 'timer' from sessionStorage to ensure that timer for total time continues from where the previous question had ended its timer.

2. It initializes and displays two timers: one for tracking total time and another for time per question. The startStop() function toggles the timer on and off, while updateTime() updates and formats both timers every second. It also stops updating when the stop flag is set to true.

VI. login.js

1. This code checks if the user is logged in and, if not, prompts login.

2. On form submission, it collects user details, registers a Firebase account, stores the information in Firestore, and saves 'user-id', 'user-name', and 'user-email' in sessionStorage.

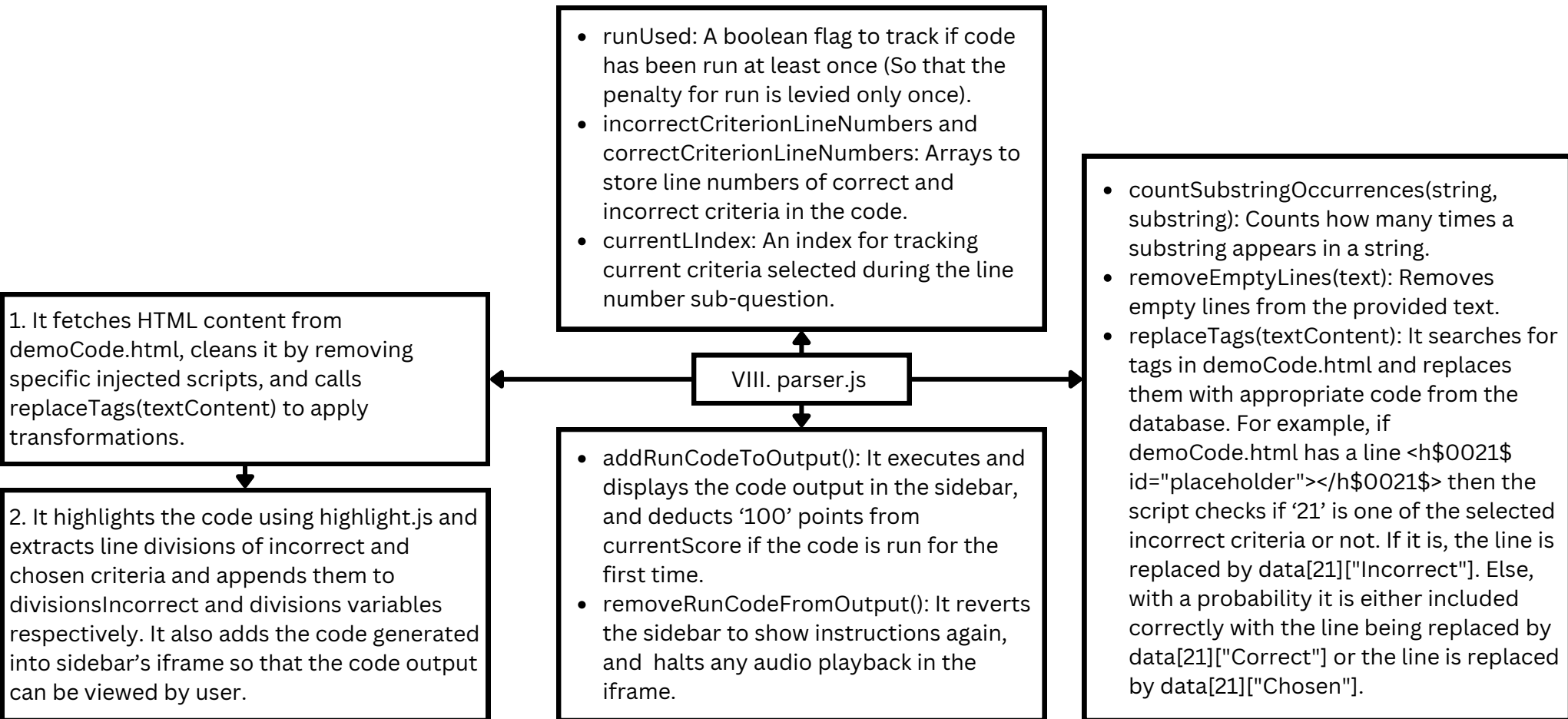
It stores the name, email, field, accessibilityKnowledge, area, subfield and timestamp (the date and time at which the user began playing the game) into a newly created document in firebase cloud DB.

1. It fetches 'info-dont-show' from sessionStorage to ensure that if the sidebar had previously been toggled off, it would stay toggled off, and vice-versa.

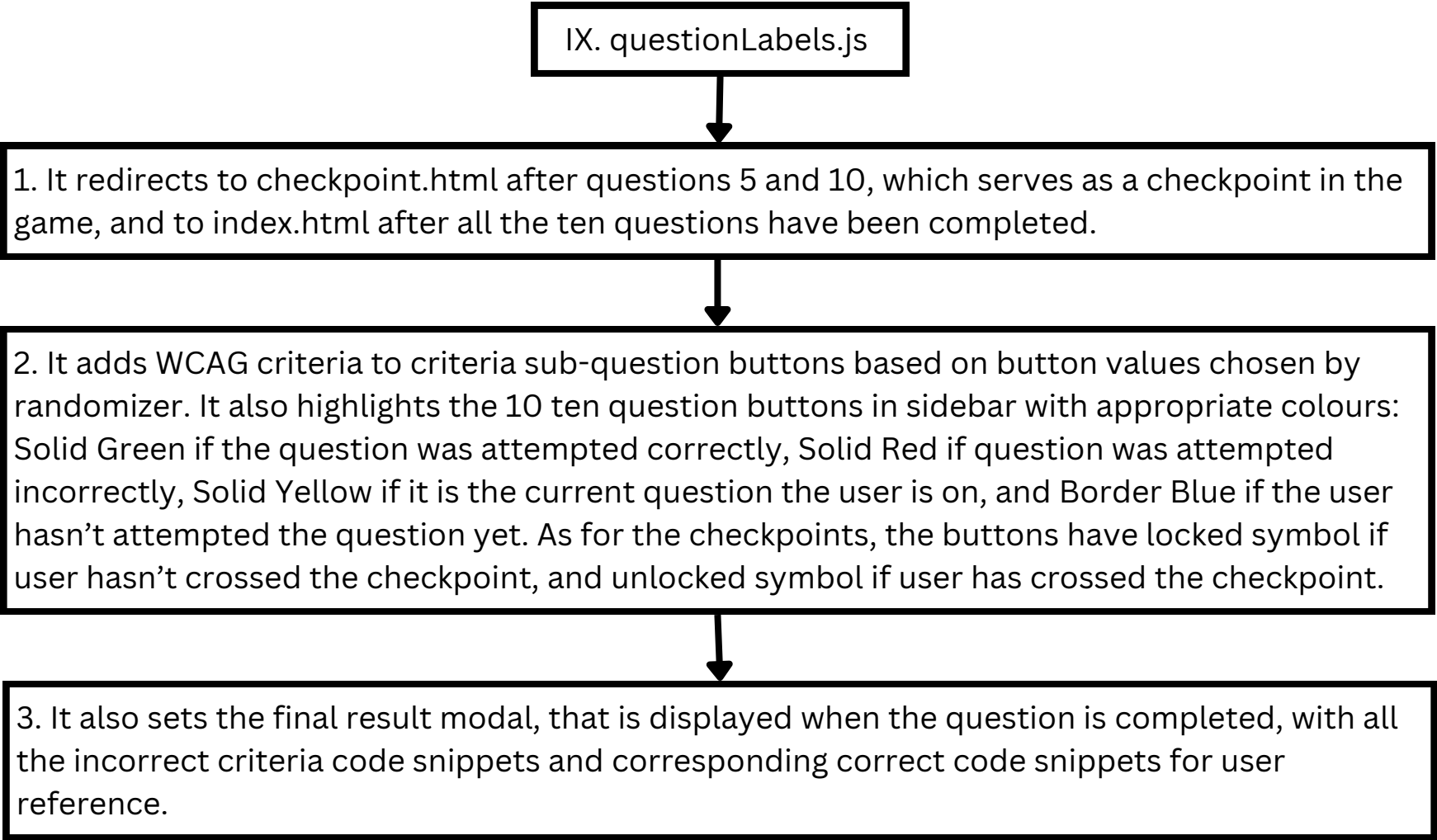
VII. toggle.js

```
graph TD; A["1. It fetches 'info-dont-show' from sessionStorage to ensure that if the sidebar had previously been toggled off, it would stay toggled off, and vice-versa."] <--> B["VII. toggle.js"]; B <--> C["• toggleSidebarOn() and toggleSidebarOff() determine the sidebar's behaviour when toggled.  
• The setInitialConditions() function ensures appropriate transition settings when the page loads, especially for smaller screens."];
```

- toggleSidebarOn() and toggleSidebarOff() determine the sidebar's behaviour when toggled.
- The setInitialConditions() function ensures appropriate transition settings when the page loads, especially for smaller screens.



IX. questionLabels.js

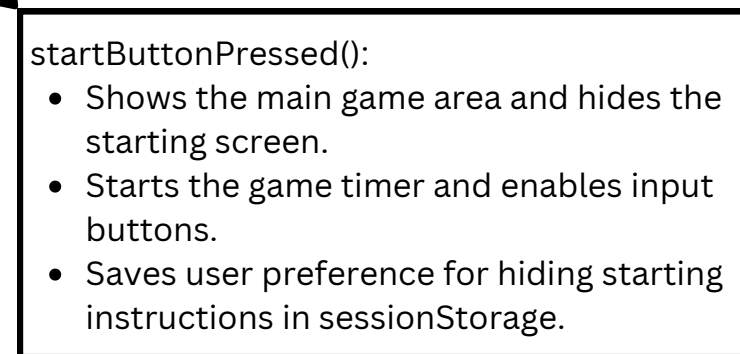
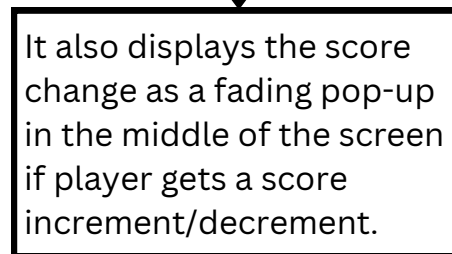
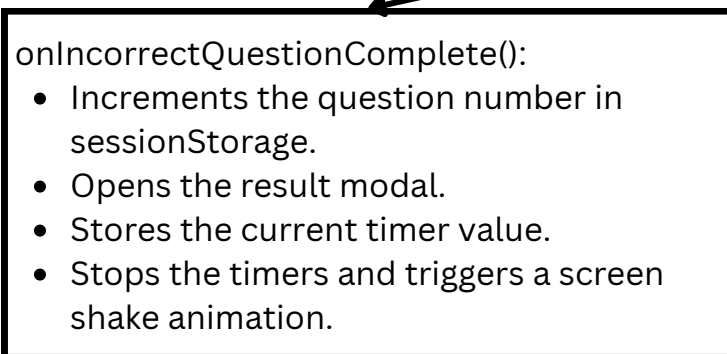
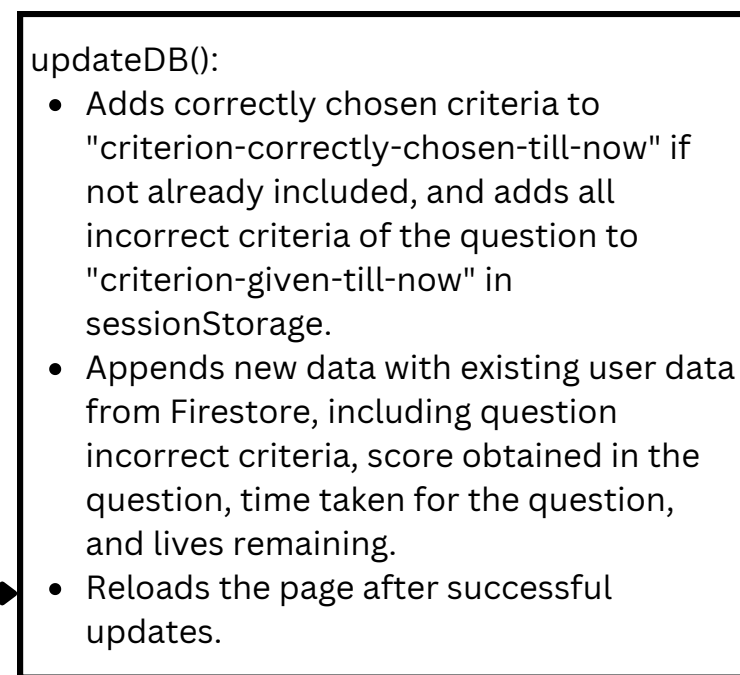
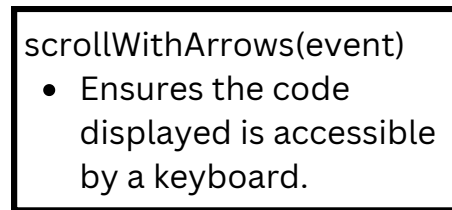
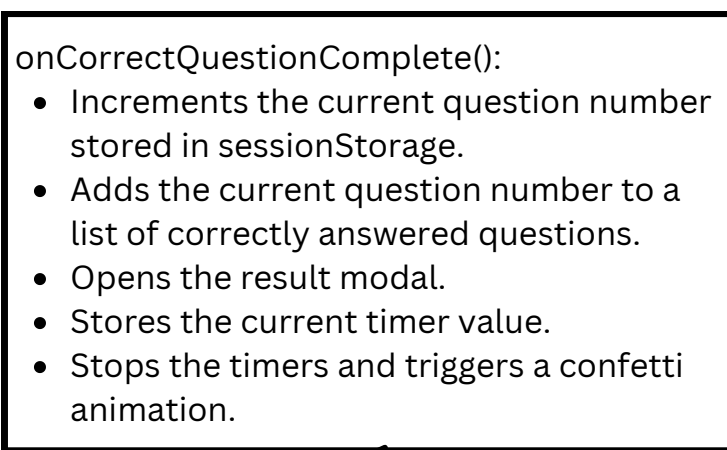


```
graph TD; A[IX. questionLabels.js] --> B[1. It redirects to checkpoint.html after questions 5 and 10, which serves as a checkpoint in the game, and to index.html after all the ten questions have been completed.]; B --> C[2. It adds WCAG criteria to criteria sub-question buttons based on button values chosen by randomizer. It also highlights the 10 ten question buttons in sidebar with appropriate colours: Solid Green if the question was attempted correctly, Solid Red if question was attempted incorrectly, Solid Yellow if it is the current question the user is on, and Border Blue if the user hasn't attempted the question yet. As for the checkpoints, the buttons have locked symbol if user hasn't crossed the checkpoint, and unlocked symbol if user has crossed the checkpoint.]; C --> D[3. It also sets the final result modal, that is displayed when the question is completed, with all the incorrect criteria code snippets and corresponding correct code snippets for user reference.];
```

1. It redirects to checkpoint.html after questions 5 and 10, which serves as a checkpoint in the game, and to index.html after all the ten questions have been completed.

2. It adds WCAG criteria to criteria sub-question buttons based on button values chosen by randomizer. It also highlights the 10 ten question buttons in sidebar with appropriate colours: Solid Green if the question was attempted correctly, Solid Red if question was attempted incorrectly, Solid Yellow if it is the current question the user is on, and Border Blue if the user hasn't attempted the question yet. As for the checkpoints, the buttons have locked symbol if user hasn't crossed the checkpoint, and unlocked symbol if user has crossed the checkpoint.

3. It also sets the final result modal, that is displayed when the question is completed, with all the incorrect criteria code snippets and corresponding correct code snippets for user reference.



X. miscellaneous.js

- `onFindingCriterion(lineNumber)`: Searches for criterion based on line number.
- `searchCriterion(lineNumber)`: Shows user the line number range selected by the user.
- `numberOfLinesInTheCriterion(i)`: Returns number of lines 'ith' incorrect code snippet has.
- `reduceBar()`: Reduces the number of bars the user has and changes the alt text accordingly. It also ends the question if WiFi bars reach 0.
- `increaseBar()`: Increases the number of bars the user has and changes alt text accordingly.
- `nextLineIndex()`: Changes the line number range the user has been displayed to the next possible line number range, which the user can enter as input to the sub-question.
- `previousLineIndex()`: Changes the line number range the user has been displayed to the previous possible line number range, which the user can enter as input to the sub-question.

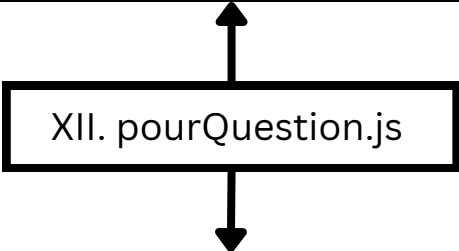
XI. `lineQuestion.js`

`onSubmitLineNumber()`:

1. This function is called when the user submits their choice of line number range as input for line number sub-question as line numbers with potential inaccessible code.
2. It checks if the criterion is indeed incorrect but has been selected by user before, and alerts the user if so.
3. If the line number range indeed has an inaccessible code snippet, it loads the part 1 success result so that users can attempt the second sub-question. It also increases the score by 20, displays the inaccessible code snippet in sidebar, highlights the line numbers in the code, and changes the question titles of sub-questions 2 and 3 according to line number range of inaccessible code snippet.
4. If the line number range doesn't an inaccessible code snippet, it loads the part 1 failure result so that users can go back to the first sub-question, and decreases the WiFi by 1 bar.

- onCorrectPour(): If user has correctly answered the POUR sub-question, then it loads the part 2 success result so user can either go to the third bonus sub-question or back to the first sub-question to identify another criterion. It also increases the score by 30 and displays number of criteria left to the user.
- onCorrectPourButQuestionComplete(): If user has correctly answered the POUR sub-question and there are no more criterion left to identify, then it loads the part 2 success result so user can either go to the third bonus sub-question or to the next question. It also increases the score by 30 and displays number of criteria left to the user.
- onWrongPour(): If user incorrectly answers the sub-question, it decreases the number of WiFi the user has and loads part 2 failure result so user can answer the sub-question again.

XII. pourQuestion.js

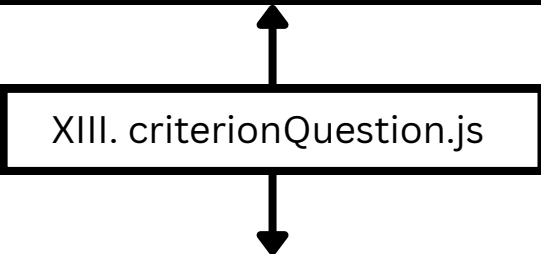


onSubmitPourQuestion(id):

1. It finds the POUR principle the inaccessible code defies, by using the first part of its criterion.
2. If this POUR principle matches with principle the user chooses, then if it wasn't the last remaining criterion it calls the onCorrectPour(), else it calls onCorrectPourButQuestionComplete() and removes criterion from the 'criterionLeftToIdentify' list.
3. If this POUR principle doesn't matches with principle the user chooses, then it calls the onWrongPour() function.

- onCorrectCriterion(): If user has correctly answered the criterion sub-question, then it loads the part 3 success result in a way that the user can go back to the first sub-question to identify another criterion. It also increases the score by 50 and increases the Wifi bars the user has left by one bar.
- onCorrectCriterionButQuestionComplete(): If user has correctly answered the criterion sub-question and there are no more criterion left to identify, then it loads the part 3 success result in a way that the user can only go to the next question. It also increases the score by 50 and increases the WiFi bars the user has left by one bar.
- onWrongCriterion(): If user incorrectly answers the sub-question, it loads part 3 failure result in a way that the user can either answer the sub-question again or skip and go to first sub-question to identify another criterion.
- onWrongCriterionButQuestionComplete(): If user incorrectly answers the sub-question, it loads part 3 failure result in a way that the user can either answer the sub-question again or skip and go to the next question.

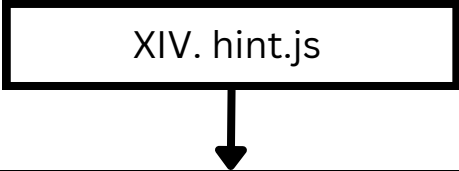
XIII. criterionQuestion.js



onSubmitCriterionQuestion(id):

1. It finds the WCAG criterion the inaccessible code defies.
2. If this criterion matches with criterion the user chooses, then if it wasn't the last remaining criterion it calls the onCorrectCriterion(), else it calls onCorrectCriterionButQuestionComplete().
3. If this POUR principle doesn't matches with principle the user chooses, then if it wasn't the last remaining criterion it calls the onWrongCriterion(), else it calls onWrongCriterionButQuestionComplete().

XIV. hint.js

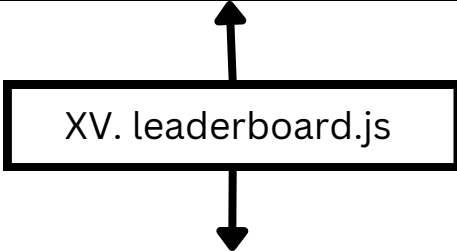


onHintClicked():

- When the user clicks the hint button, then it chooses the first criterion in the 'criterionLeftToIdentify' list and adds it to 'hintedCriteria' list.
- It then replaces the corresponding incorrect code snippet with the commented code snippet from the database so user is presented with a comment as hint in code displayed.
- It also autofills the hinted code snippet line number range in input of the first sub-question, to make the first sub-question easier.
- It also disables the hint button till the user utilizes this hint.

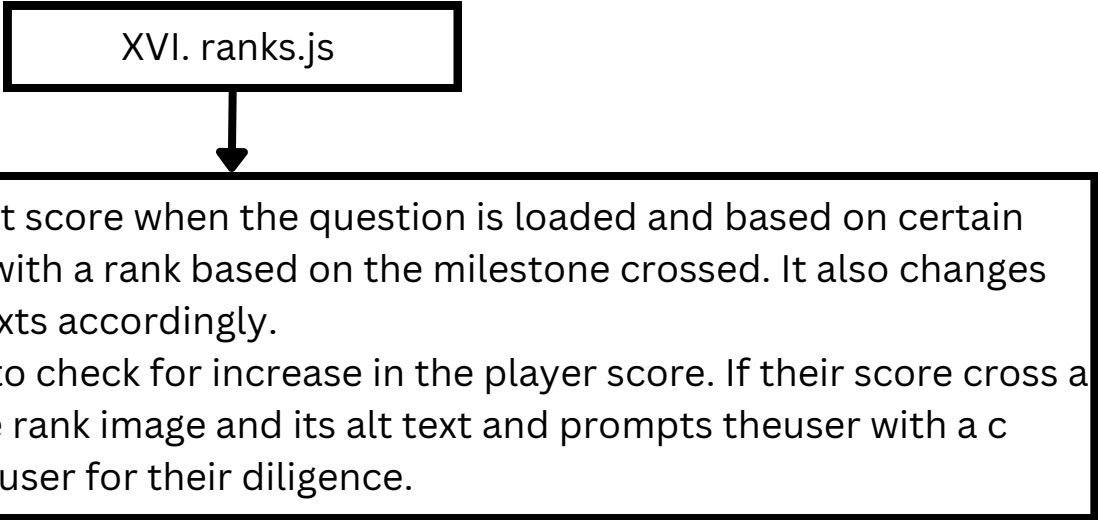
1. It loads the default player leaderboard based on all entries and no filters applied.
2. Then it adds eventListeners to time filter and radio buttons and calls `changeLeaderboard(fieldOptionChosen, radioChosen)` so that the leaderboard is changed according to the filters selected by the user.

XV. leaderboard.js



- `addEntry(number, name, score, timeTaken)`: It generates a styled list item (li) representing a player's rank, name, score, and time taken. Depending on the rank (number), it displays medals for the top three positions.
- `writeIntoLeaderboard(players)`: It clears the existing leaderboard and populates it with the top 10 players from a provided list, or fills any empty slots with placeholders.
- `filterPlayersPlayedToday(players)`, `filterPlayersPlayedPastWeek(players)`, `filterPlayersPlayedPastMonth(players)`, `filterPlayersPlayedPastSixMonths(players)`, `filterPlayersPlayedPastYear(players)`: They are defined to filter players based on when they last played, covering timeframes from “Today” to “Past Year.”
- `filterBasedOnRadio(players, radioChosen)`: It filters players by user categories (e.g., all users, or specific user types such as "Working Professional"). The `radioChosen` parameter determines which user category to filter. These are then called by `filterBasedOnTime(selectedleaderboardFilterField)` based on the time filter chosen.
- `changeLeaderBoard(fieldOptionChosen, radioChosen)`: It applies both time-based and user-type filters, combining the results and updating the leaderboard display with `writeIntoLeaderboard(players)` function.

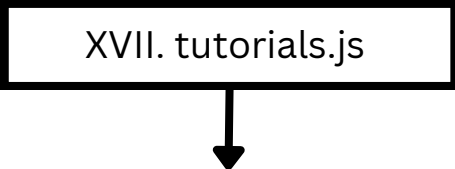
XVI. ranks.js



```
graph TD; A[XVI. ranks.js] --> B[1. First it checks the user's current score when the question is loaded and based on certain milestones, it assigns the user with a rank based on the milestone crossed. It also changes the rank images and their alt texts accordingly.  
2. Then it observes the score div to check for increase in the player score. If their score cross a milestone, then it increases the rank image and its alt text and prompts the user with a congrats modal, applauding the user for their diligence.];
```

1. First it checks the user's current score when the question is loaded and based on certain milestones, it assigns the user with a rank based on the milestone crossed. It also changes the rank images and their alt texts accordingly.
2. Then it observes the score div to check for increase in the player score. If their score cross a milestone, then it increases the rank image and its alt text and prompts the user with a congrats modal, applauding the user for their diligence.

XVII. tutorials.js



```
graph TD; A[XVII. tutorials.js] --> B[onTutorialReset(): It initiates an interactive tutorial for a UI interface using the introJs library. The function checks if :main-start-area" is displayed. If visible, it starts a step-by-step guide with descriptions for UI elements related the code-based game after its code has been displayed. If not, it provides an alternate tutorial tailored for UI elements related to when the game hasn't been started yet, and game is still showing the initial game instructions.]; B --> C[Each step of the tutorial contains a JSON with keys 'element' that has the id of html element to be explained, 'title' which has the name of the element as will be displayed to the user, and the 'intro' which contains a short explanation of the element's role in the game. When the user clicks the next button, introJS switches from the current step to the next step, and so the UI tutorial progresses.];
```

onTutorialReset(): It initiates an interactive tutorial for a UI interface using the introJs library. The function checks if " :main-start-area" is displayed. If visible, it starts a step-by-step guide with descriptions for UI elements related the code-based game after its code has been displayed. If not, it provides an alternate tutorial tailored for UI elements related to when the game hasn't been started yet, and game is still showing the initial game instructions.

Each step of the tutorial contains a JSON with keys 'element' that has the id of html element to be explained, 'title' which has the name of the element as will be displayed to the user, and the 'intro' which contains a short explanation of the element's role in the game. When the user clicks the next button, introJS switches from the current step to the next step, and so the UI tutorial progresses.

Data Collected in Firebase

accessibilityKnowledge: "Proficient"

area: "Asia"

correctlyAnswered

- 0 true
- 1 true

User is from 'Asia' and 'Proficient' in Accessibility.

Q1 answered correctly.
Q2 answered correctly.

email: "rohanjha@aol.in"

field: "Undergraduate Student"

finalScore: 2110

hintedCriteriaList

- 0 ""
- 1 "[8,30]"

Email is 'rohanjha@aol.in'.

User is an Undergraduate student and has a final score of 2110 in the game.

User took no hints in Q1 and two hints in Q2 for criteria 8 and 30 from the database.

livesRemaining

- 0 "Four"
- 1 "One"

User had 4 lives left in Q1.
User had 1 life left in Q2.

name: "Rohan Jha"

questionCriterion

- 0 "[18]"
- 1 "[8,30]"

User name is 'Rohan Jha'.

The game displayed criterion 18 from DB in Q1 and criteria 8 and 30 in Q2.

score

- 0 50
- 1 100

User got a total score of 50 points in Q1 and 100 points in Q2. By the end of Q2, user had 150 points.

subfield: "BE Computer Science"

timeTaken

- 0 "Question Timer :: 00:00:13"
- 1 "Question Timer :: 00:00:16"

User is in 'BE Computer Science' course. The user took 13 and 16 seconds for Q1 and Q2 respectively.

timestamp: "Wed Nov 06 2024 05:03:08 GMT+0530 (India Standard Time)"

totalTimeTaken: "00:03:41"

unpickedCriterion

- 0 ""
- 1 ""

User started the game on 6 Nov 2024 05:03 IST and took 3m 41s to complete it.

User had no criteria unpicked in Q1 and Q2.